Computer Architecture
CS 622 / EE 520
Fall 2026

Lecture 19

Shahid Masud

Topics

- Caches
  - Performance
  - Solved Questions
  - Cache Optimization Techniques

Quiz Next Week

Further Readings

Reading Notes

Readings:
Appendix B of course textbook

Slide credits

Lecture 11: Memory Hierarchy
-- Cache Organization and Performance

CSCE 513 Computer Architecture

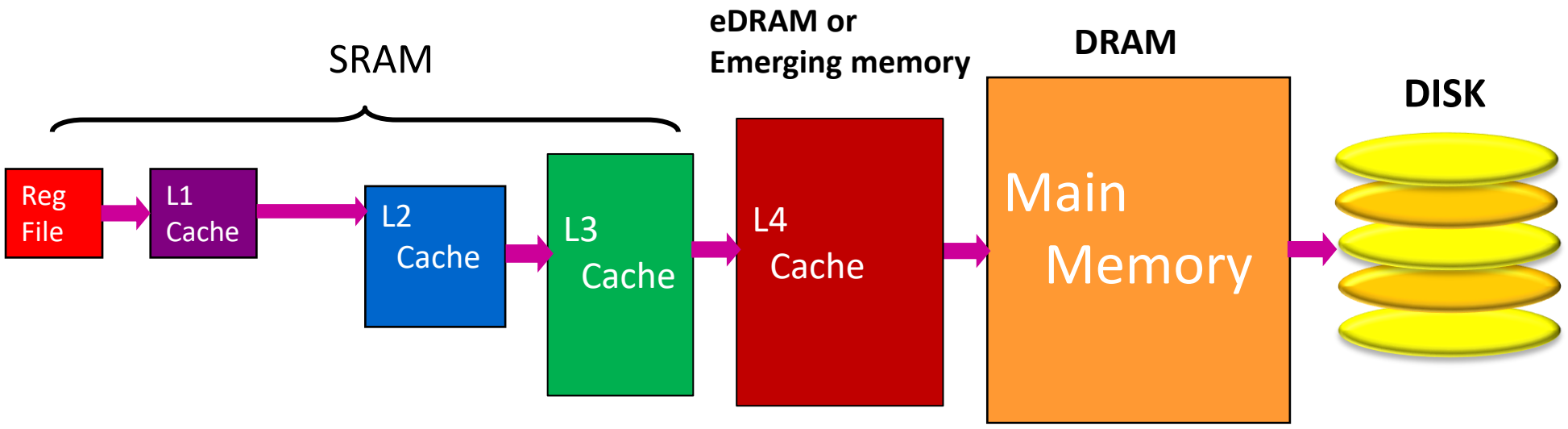
Department of Computer Science and Engineering
Yonghong Yan
yanyh@cse.sc.edu
https://passlab.github.io/CSCE513

Slides adapted from:
https://www.cs.nthu.edu.tw/~king/courses/cs5100/2014/N04-Cache-1.pdf

Cache Hit, Miss, and Replacement, Some Examples

Memory Hierarchy Design 1

- Entire addressable memory space available in largest, slowest memory
- Incrementally smaller and faster memories, each containing a subset of the memory below it, proceed in steps up toward the processor



Replacement Policy: NRU, Others

- For 2-way set-associative cache
  - 1-bit per set indicates LRU/MRU; set/clear on each access
- For >2-way, LRU is difficult/expensive in HW
  - Need to maintain history of access order
- Not recently used (NRU)
  - A "*used bit*" is associated with every block, initially 0
  - Whenever a block is accessed, its used bit is set to 1
    - If on an access, all other used bits in a set are 1, they are reset to 0 except the bit of the block that is accessed
  - A *replacement pointer* is used to look for a block whose used bit is 0 to be replaced
  - Used by Intel Itanium, Sparc T2
- Other policies: Pseudo LRU, LRU Insertion Policy (LIP)

Questions for Cache Hit and Miss (Done last lecture)

- ***Block placement***: where in upper hierarchy level?
  - fully associative, direct mapped, set-associative
- ***Block identification***: find block in upper hierarchy level?
  - search and match address tag:
    block address (= tag + index) + block offset
  - valid bit: tag-index boundary and associativity
- ***Block replacement***: which block to replace on miss?
  - easy for direct map; random/LRU for associative
- ***Write strategy***: what happens on a write?
  - Write through and write back
  - Write allocate and not allocate

Write Miss Policy – Allocate vs No Allocate

- Two options on a write miss
  - Write allocate – the block is allocated on a write miss, followed by the write hit actions.
    - Write misses act like read misses.
  - No-write allocate – write misses do not affect the cache. The block is modified only in the lower-level memory.
    - Block stay out of the cache in no-write allocate until the program tries to read the blocks, but with write allocate even blocks that are only written will still be in the cache.

What is Write Allocate?

- What should happen on a write miss?
- Alternatives for write-through
  - Allocate on miss: fetch the block
  - Write around: don't fetch the block
    - Since programs often write a whole block before reading it (e.g., initialization)
- For write-back
  - Usually fetch the block

Example of Write Miss Policy

- Example: Assume a fully associative write-back cache with many cache entries that starts empty. Below is sequence of five memory operations (The address is in square brackets):

  Write Mem[100];
  Write Mem[100];
  Read Mem[200];
  Write Mem[200];
  Write Mem[100].

  What are the number of hits and misses (inclusive reads and writes) when using no-write allocate versus write allocate?

- *Answer*

No-write Allocate:

Write Mem[100]; 1 write miss
Write Mem[100]; 1 write miss
Read Mem[200]; 1 read miss
Write Mem[200]; 1 write hit
Write Mem[100]. 1 write miss
4 misses; 1 hit

Write allocate:

Write Mem[100]; 1 write miss
Write Mem[100]; 1 write hit
Read Mem[200]; 1 read miss
Write Mem[200]; 1 write hit
Write Mem[100]; 1 write hit
2 misses; 3 hits

Cache and CPU Performance

Memory Hierarchy Performance - AMAT

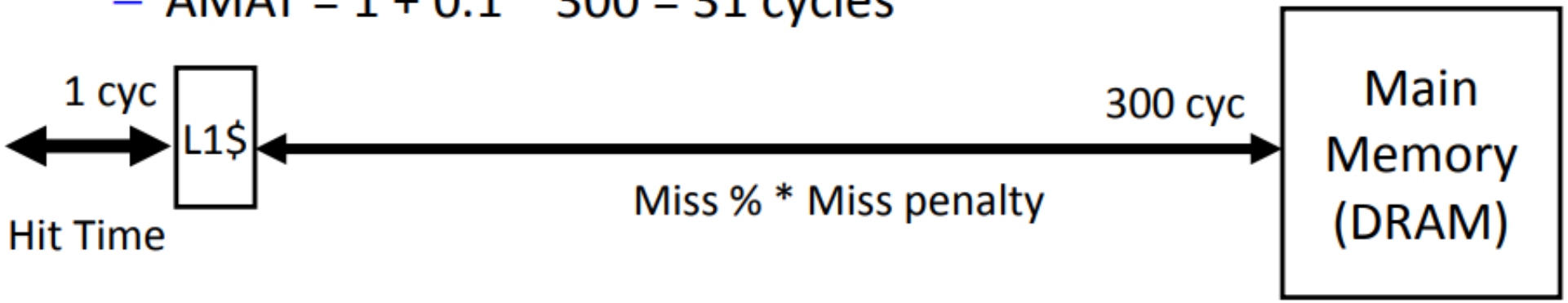
- Average memory access time between levels
  = Hit time + Miss rate x Miss penalty
- **Miss penalty**: time to fetch a block from lower memory level
  - **Access time**: time to lower level; function of latency
  - **Transfer time**: function of bandwidth between levels
    - Transfer one “cache line/block” at a time
    - Transfer at the size of the memory-bus width

Cache and CPU Performance

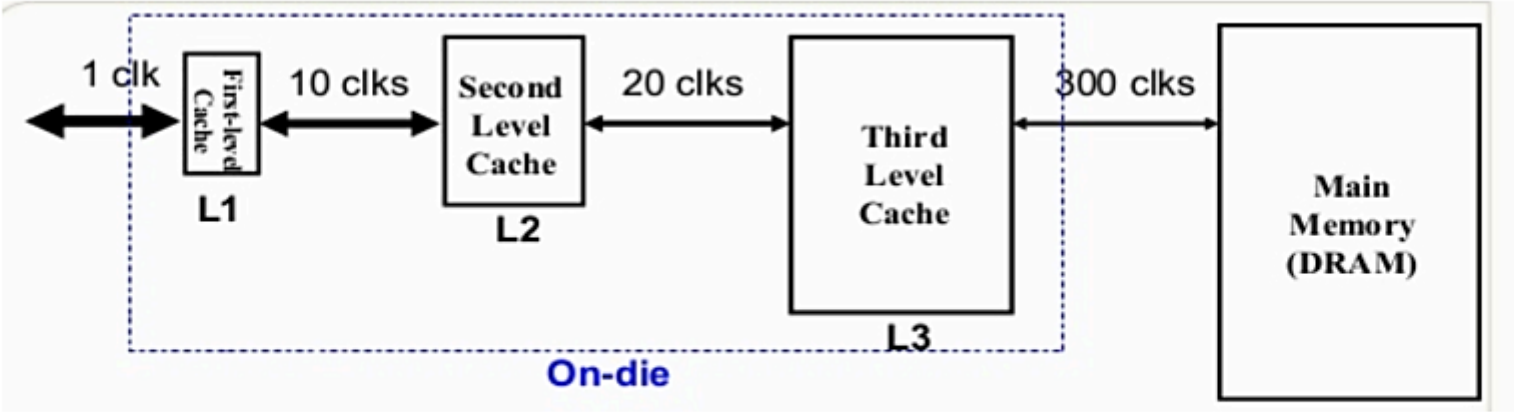
- **CPU time** = (CPU cycle + memory stall cycles) x clock cycle time
- **Memory stall cycles** = memory access x miss rate x miss penalty
  - Assumptions: memory stalls are due to cache misses, hit cycles included in CPU execution cycles, reads and writes have same miss rate and penalty
- **CPU time** = instruction count x (CPI_{ideal} + memory access/instruction x miss rate x miss penalty) x clock cycle time
- Cache design becomes more important for CPUs with lower CPI and higher clock rates

Average Memory Access Time

- Average Memory Access Time (AMAT)
  - = Hit Time + Miss rate * Miss Penalty
  - = T_{hit}(L1) + Miss%(L1) * T(memory)
- Example:
  - Cache Hit = 1 cycle
  - Miss rate = 10% = 0.1
  - Miss penalty = 300 cycles
  - AMAT = 1 + 0.1 * 300 = 31 cycles



AMAT For Multilevel Cache



- Average Memory Access Time (AMAT)
  = **Hit Time + Miss Rate * Miss Penalty**

Miss Penalty(L3) = T(Mem)

Miss Penalty(L2) = AMAT(L3) = T_{hit}(L3) + Miss%(L3) * Miss Penalty(L3)

Miss Penalty(L1) = AMAT(L2) = T_{hit}(L2) + Miss%(L2) * Miss Penalty(L2)

AMAT = T_{hit}(L1) + Miss%(L1) * Miss Penalty(L1)

= T_{hit}(L1) + Miss%(L1) * {T_{hit}(L2) + Miss%(L2) * Miss Penalty(L2)}

= T_{hit}(L1) + Miss%(L1) * {T_{hit}(L2) + Miss%(L2) *

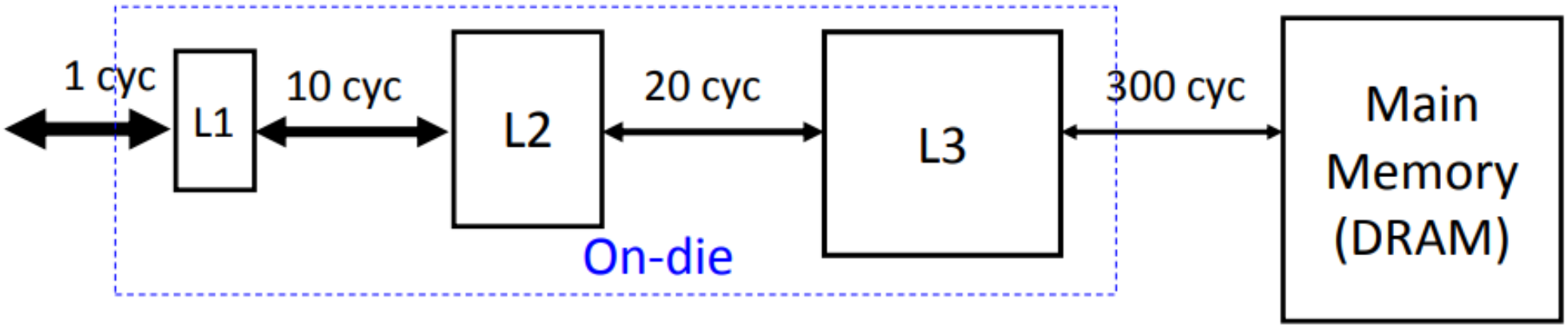
[T_{hit}(L3) + Miss%(L3) * *T(Mem)*] }

AMAT Example of Multilevel Cache

- AMAT = T_{hit}(L1) + Miss%(L1)* (T_{hit}(L2) + Miss%(L2)* (T_{hit}(L3) + Miss%(L3)*T(memory)))
- Example:
  - Miss rate L1=10%, T_{hit}(L1) = 1 cycle
  - Miss rate L2=5%, T_{hit}(L2) = 10 cycles
  - Miss rate L3=1%, T_{hit}(L3) = 20 cycles
  - T(memory) = 300 cycles

14.7x speed-up!

AMAT = 2.115 cyc



Multilevel Cache Example – Effective CPI

Multilevel Cache Example

- Given
  - CPU base CPI = 1, clock rate = 4GHz
  - Miss rate/instruction = 2%
  - Main memory access time = 100ns
- With just primary cache
  - Miss penalty = 100ns/0.25ns = 400 cycles
  - Effective CPI = 1 + 0.02 × 400 = 9

Multilevel Cache Example - contd

Example (cont.)

- Now add L-2 cache
  - Access time = 5ns
  - Global miss rate to main memory = 0.5%
- Primary miss with L-2 hit
  - Penalty = 5ns/0.25ns = 20 cycles
- Primary miss with L-2 miss
  - Extra penalty = 500 cycles
- CPI = 1 + 0.02 × 20 + 0.005 × 400 = 3.4
- Performance ratio = 9/3.4 = 2.6

Cache Performance and CPI

- Given
  - I-cache miss rate = 2%
  - D-cache miss rate = 4%
  - Miss penalty = 100 cycles
  - Base CPI (ideal cache) = 2
  - Load & stores are 36% of instructions

- Miss cycles per instruction
  - I-cache: 0.02 × 100 = 2
  - D-cache: 0.36 × 0.04 × 100 = 1.44

- Actual CPI = 2 + 2 + 1.44 = 5.44
  - Ideal CPU is 5.44/2 =2.72 times faster

Main Memory Supporting Caches

- Use DRAMs for main memory
  - Fixed width (e.g., 1 word)
  - Connected by fixed-width clocked bus
    - Bus clock is typically slower than CPU clock
- Example cache block read
  - 1 bus cycle for address transfer
  - 15 bus cycles per DRAM access
  - 1 bus cycle per data transfer
- For 4-word block, 1-word-wide DRAM
  - Miss penalty = 1 + 4×15 + 4×1 = 65 bus cycles
  - Bandwidth = 16 bytes / 65 cycles = 0.25 B/cycle

Memory Hierarchy Impact on CPI Performance

- Suppose a processor executes at
  - Clock Rate = 200 MHz (5 ns per cycle)
  - CPI = 1.1
  - 50% arith/logic, 30% ld/st, 20% control
- Suppose that 10% of memory operations get 50 cycle miss penalty
- Suppose that 1% of instruction accesses get same miss penalty

Ideal CPI	1.1
Data Miss	1.5
Inst Miss	0.5

- CPI = ideal CPI + average stalls per instruction

  = 1.1(cycles/ins)

  + [**0.30** (DataMops/ins) x 0.10 (miss/DataMop) x 50 (cycle/miss)]

  + [**1** (InstMop/ins) x 0.01 (miss/InstMop) x 50 (cycle/miss)]

  = (1.1 + 1.5 + .5) cycle/ins = 3.1

- **2/3.1 (64.5%) of the time the proc is stalled waiting for memory!**

Basic Cache Optimization

Memory Hierarchy Performance Equations

- Two indirect performance measures have waylaid many a computer designer.
  - ***Instruction count*** **is independent of the hardware;**
  - ***Miss rate*** **could be independent of the hardware mostly**

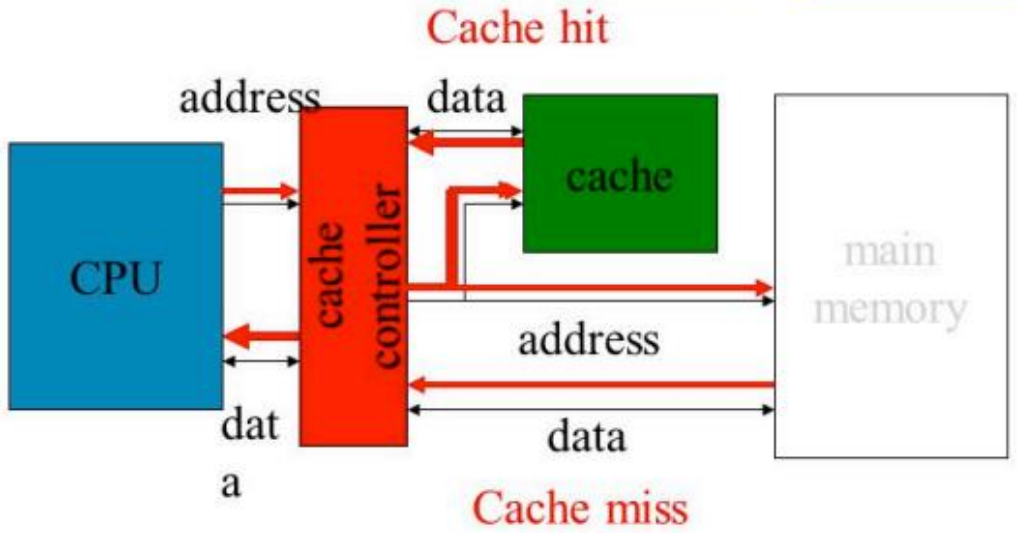
$$CPU\ Time = IC * \left(CPI_{Execution} + \frac{Memory\ Accesses}{Instruction} \times Miss\ Rate \times Miss\ Penalty\right) \times Clock\ Cycle\ Time$$

- A better measure of memory hierarchy performance is the ***Average Memory Access Time*** **(AMAT) per instructions**

$$AMAT = Hit\ time + Miss\ rate \times Miss\ penalty$$

Improving Cache Performance - Parameters

Average Memory Access Time = Hit Time + Miss Rate * Miss Penalty



Goals	Basic Approaches
Reducing Miss Rate	Larger block size, larger cache size and higher associativity
Reducing Miss Penalty	Multilevel caches, and higher read priority over writes
Reducing Hit Time	Avoid address translation when indexing the cache

Six Basic Cache Optimization Techniques

- **1. Larger block size**
  - Exploit spatial locality to reduce compulsory misses
  - Increase capacity and conflict misses, increase miss penalty
- **2. Larger total cache capacity**
  - Reduce capacity misses
  - Increase hit time, increase power consumption
- **3. Higher associativity**
  - Reduce conflict misses
  - Increase hit time, increase power consumption
- **4. Multilevel Caches**
- **5. Giving priority to read misses over writes**
  - Check WB on read; if no conflicts, read bypass writes in WB
- **6. Avoiding Address Translation in Cache Indexing**

Much Larger Block Size -> Increases Miss Rate

	Cache size			
Block size	4K	16K	64K	256K
16	8.57%	3.94%	2.04%	1.09%
32	7.24%	2.87%	1.35%	0.70%
64	7.00%	2.64%	1.06%	0.51%
128	7.78%	2.77%	1.02%	0.49%
256	9.51%	3.29%	1.15%	0.49%

Figure B.11 Actual miss rate versus block size for the five different-sized caches in Figure B.10. Note that for a 4 KB cache, 256-byte blocks have a higher miss rate than 32-byte blocks. In this example, the cache would have to be 256 KB in order for a 256-byte block to decrease misses.

1. Reduce Miss Rate via Large Block Size

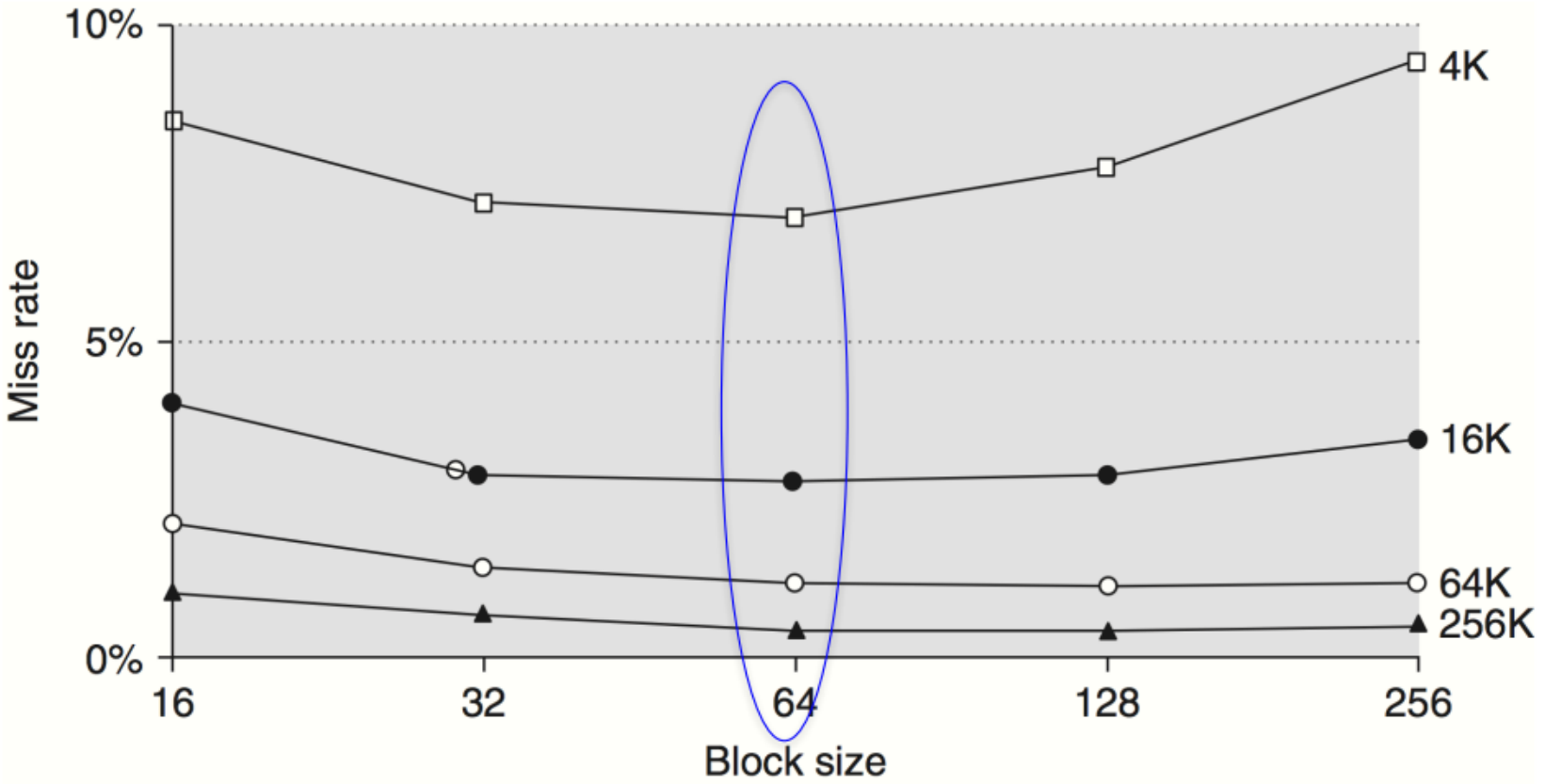
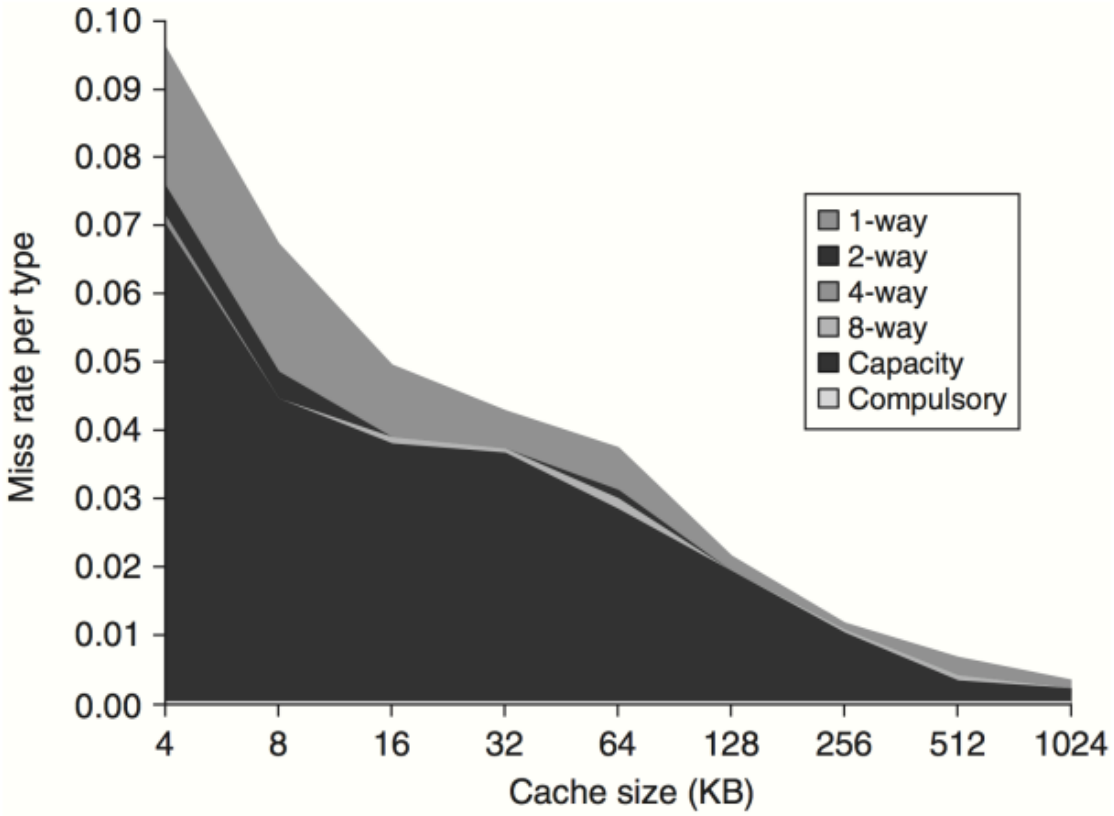


Figure B.10 Miss rate versus block size for five different-sized caches. Note that miss rate actually goes up if the block size is too large relative to the cache size. Each line represents a cache of different size. Figure B.11 shows the data used to plot these lines. Unfortunately, SPEC2000 traces would take too long if block size were included, so

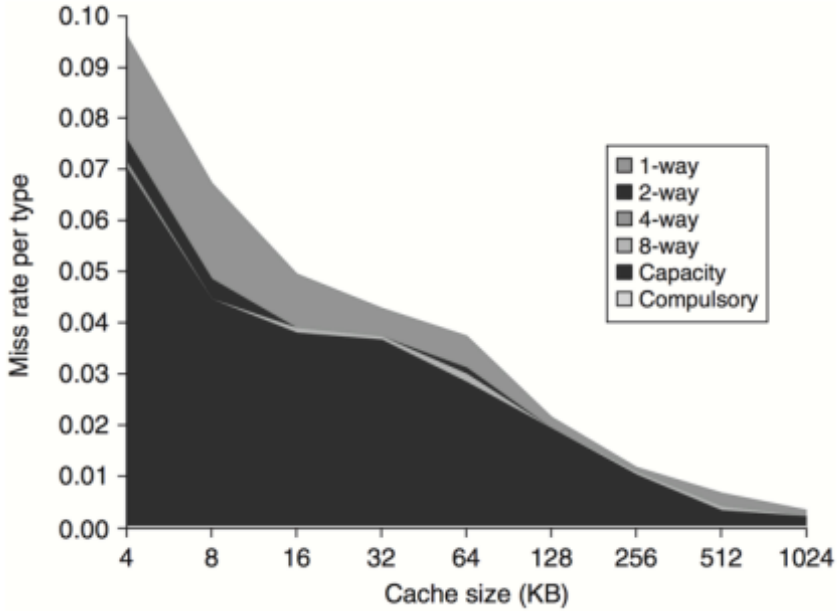
2. Reduce Miss Rate via Large Cache Size

- Increasing capacity of cache reduces capacity misses
- May be longer hit time and higher cost
- Trends: Larger L2 or L3 off-chip caches



3. Reduce Miss Rate via Higher Associativity

- 2:1 Cache Rule:
  - Miss Rate DM cache size N = Miss Rate 2-way cache size N/2
- 8-way set associative is as effective as fully associative for practical purposes
- Tradeoff: higher associative cache complicates the circuit
  - May have longer clock cycle
- Beware: Execution time is the only final measure!
  - Will Clock Cycle time increase?
  - Hill [1988] suggested hit time for 2-way vs. 1-way external cache +10%, internal + 2%



4. Reduce Miss Penalty via Multilevel Caches

- Approaches
  - Make the cache faster to keep pace with the speed of CPUs
  - Make the cache larger to overcome the widening gap
- **L1: fast hits, L2: fewer misses**
- L2 Equations

$$\text{Average memory access time} = \text{Hit time}_{L1} + \text{Miss rate}_{L1} \times \text{Miss penalty}_{L1}$$

and

$$\text{Miss penalty}_{L1} = \text{Hit time}_{L2} + \text{Miss rate}_{L2} \times \text{Miss penalty}_{L2}$$

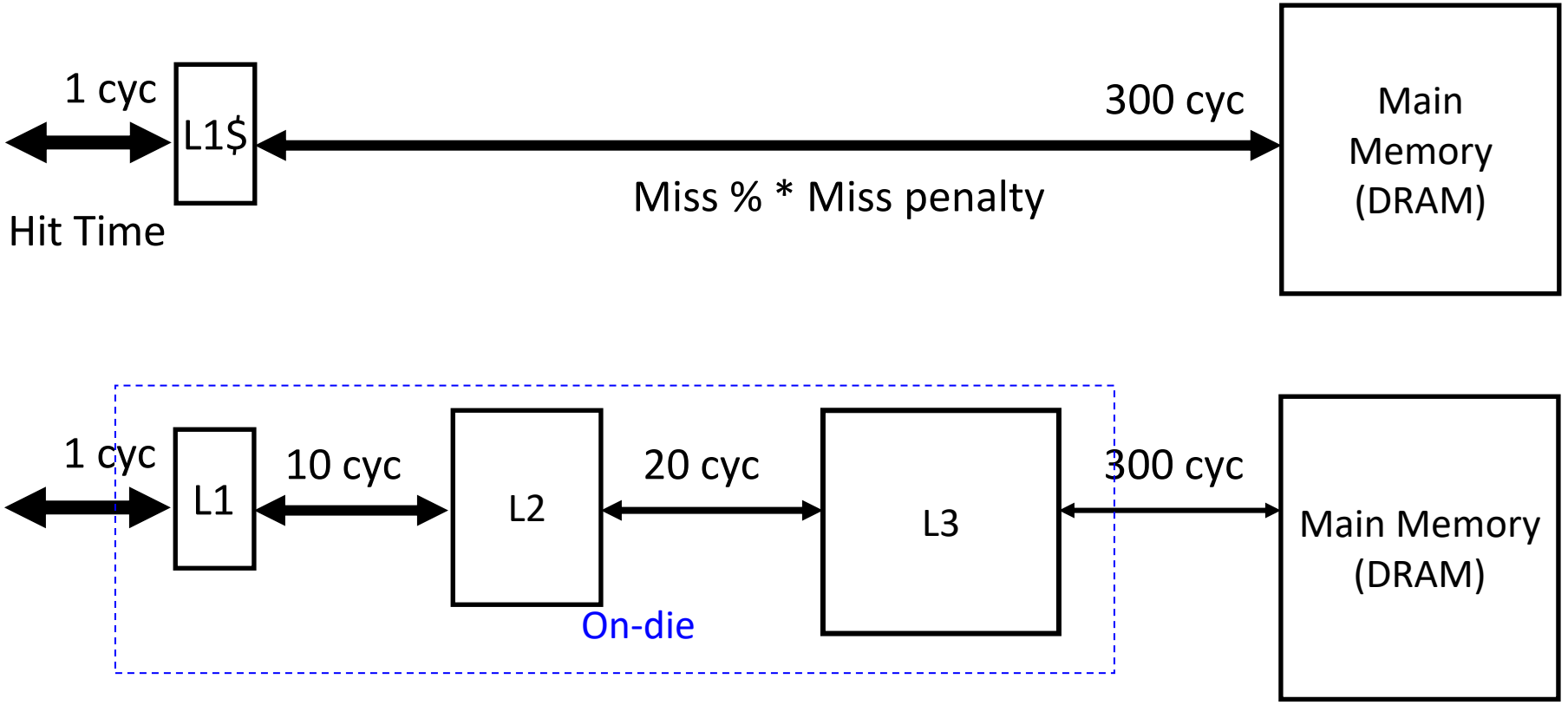
so

$$\text{Average memory access time} = \text{Hit time}_{L1} + \text{Miss rate}_{L1} \times (\text{Hit time}_{L2} + \text{Miss rate}_{L2} \times \text{Miss penalty}_{L2})$$

- $\text{Hit Time}_{L1} << \text{Hit Time}_{L2} << \ldots << \text{Hit Time}_{Mem}$
- $\text{Miss Rate}_{L1} < \text{Miss Rate}_{L2} < \ldots$

Adding More Cache Levels

- More cache levels



Miss Rate In Multilevel Caches

- **Local miss rate**— misses in this cache divided by the total number of memory accesses to this cache (Miss rateL1 , Miss rateL2)
  - L1 cache skims the cream of the memory accesses
- **Global miss rate**—misses in this cache divided by the total number of memory accesses generated by the CPU (Miss rateL1, Miss RateL1 x Miss RateL2)
  - Indicate what fraction of the memory accesses that leave the CPU go all the way to memory

Miss Rate Analysis in Multilevel Caches

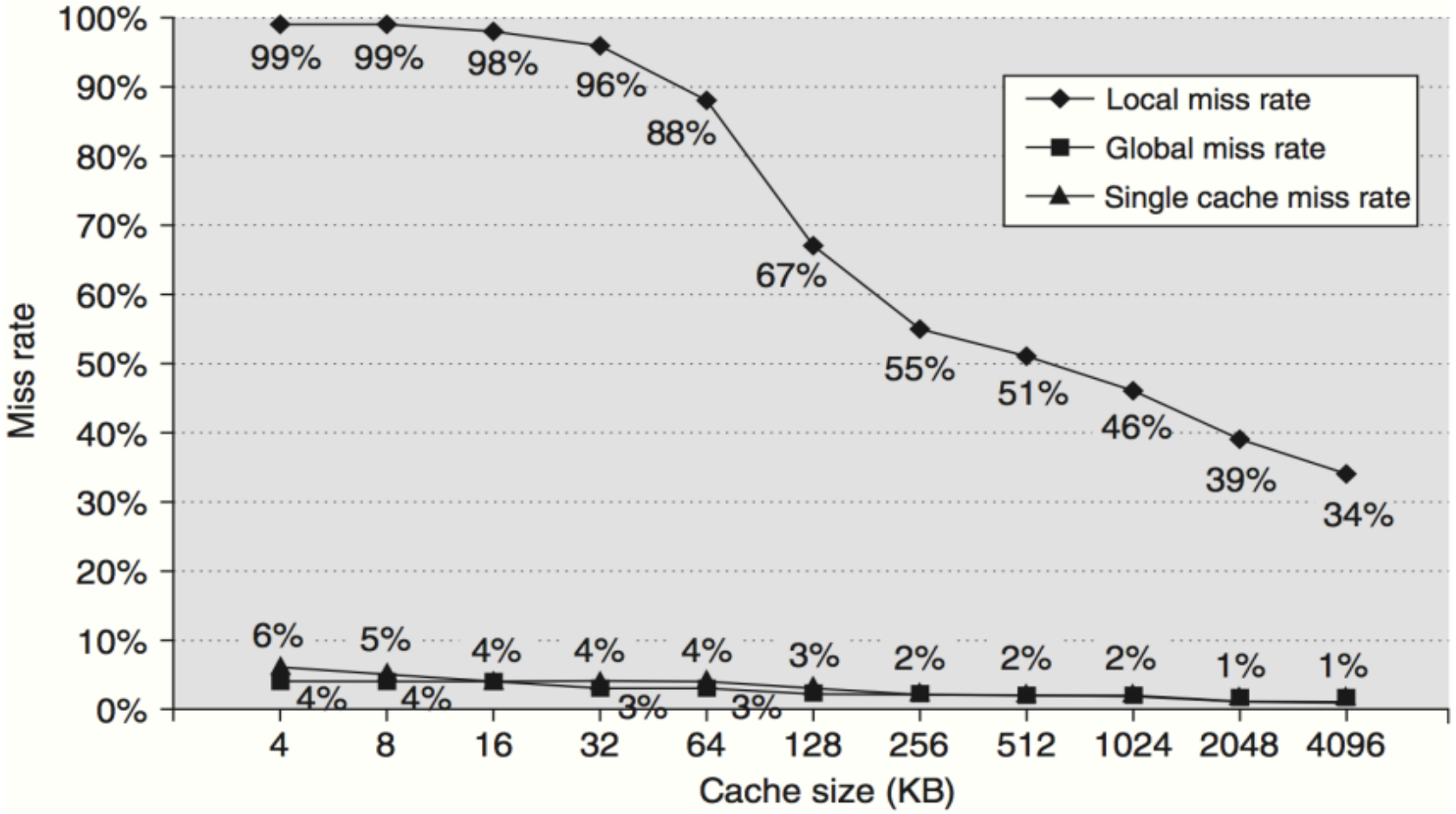


Figure B.14 Miss rates versus cache size for multilevel caches. Second-level caches *smaller* than the sum of the two 64 KB first-level caches make little sense, as reflected in the high miss rates. After 256 KB the single cache is within 10% of the global miss rates. The miss rate of a single-level cache versus size is plotted against the local miss rate and

Design Aspects of L2 Cache

- Size
  - Since everything in L1 cache is likely to be in L2 cache, L2 cache should be much bigger than L1
- Whether data in L1 is in L2
  - novice approach: design L1 and L2 independently
  - multilevel inclusion: L1 data are always present in L2
    - Advantage: easy for consistency between I/O and cache (checking L2 only)
    - Drawback: L2 must invalidate all L1 blocks that map onto the 2nd-level block to be replaced => slightly higher 1st-level miss rate
      - i.e. Intel Pentium 4: 64-byte block in L1 and 128-byte in L2
  - multilevel exclusion: L1 data is never found in L2
    - A cache miss in L1 results in a swap of blocks between L1 and L2
    - Advantage: prevent wasting space in L2
      - i.e. AMD Athlon: 64 KB L1 and 256 KB L2

5. Reduce Miss Penalty by Giving Priority to Read Misses over Writes

- Serve reads before writes have been completed
- Write through with write buffers

```
SW    R3, 512(R0)     ; M[512] <- R3      (cache index 0)
LW    R1, 1024(R0)    ; R1 <- M[1024]     (cache index 0)
LW    R2, 512(R0)     ; R2 <- M[512]      (cache index 0)
```

  Problem: write through with write buffers offer RAW conflicts with main memory reads on cache misses
  - If simply wait for write buffer to empty, might increase read miss penalty (old MIPS 1000 by 50%)
  - Check write buffer contents before read; if no conflicts, let the memory access continue
- Write Back

  Suppose a read miss will replace a dirty block
  - Normal: Write dirty block to memory, and then do the read
  - Instead: Copy the dirty block to a write buffer, do the read, and then do the write
  - CPU stall less since restarts as soon as do read

6. Reduce Hit Time by Avoiding Address Translation during Indexing of the Cache

- Importance of cache hit time
  - Average Memory Access Time = **Hit Time** + (Miss Rate * Miss Penalty)
  - More importantly, cache access time **limits the clock cycle rate** in many processors today!
- Fast hit time:
  - Quickly and efficiently find out if data is in the cache, and
  - if it is, get that data out of the cache
- Four techniques:
  1. Small and simple caches
  2. Avoiding address translation during indexing of the cache
  3. Pipelined cache access
  4. Trace caches

Summary of 6 Basic Cache Optimization Techniques

Summary of the 6 Basic Cache Optimization Techniques (Textbook B.3)

Technique	Hit time	Miss penalty	Miss rate	Hardware complexity	Comment
Larger block size		–	+	0	Trivial; Pentium 4 L2 uses 128 bytes
Larger cache size	–		+	1	Widely used, especially for L2 caches
Higher associativity	–		+	1	Widely used
Multilevel caches		+		2	Costly hardware; harder if L1 block size ≠ L2 block size; widely used
Read priority over writes		+		1	Widely used
Avoiding address translation during cache indexing	+			1	Widely used

Figure B.18 Summary of basic cache optimizations showing impact on cache performance and complexity for the techniques in this appendix. Generally a technique helps only one factor. + means that the technique improves the factor, – means it hurts that factor, and blank means it has no impact. The complexity measure is subjective, with 0 being the easiest and 3 being a challenge.

Memory Hierarchy Performance

- Two indirect performance measures have waylaid many a computer designer.
  - ***Instruction count*** **is independent of the hardware;**
  - ***Miss rate*** **could be independent of the hardware mostly**

$$CPU\ Time = IC * \left(CPI_{Execution} + \frac{Memory\ Accesses}{Instruction} \times Miss\ Rate \times Miss\ Penalty\right) \times Clock\ Cycle\ Time$$

- A better measure of memory hierarchy performance is the ***Average Memory Access Time*** **(AMAT) per instructions**

$$AMAT = Hit\ time + Miss\ rate \times Miss\ penalty$$

Example: Miss Rate vs Reduce AMAT

Example Figure B.11 shows the actual miss rates plotted in Figure B.10. Assume the memory system takes 80 clock cycles of overhead and then delivers 16 bytes every 2 clock cycles. Thus, it can supply 16 bytes in 82 clock cycles, 32 bytes in 84 clock cycles, and so on. Which block size has the smallest average memory access time for each cache size in Figure B.11?

Answer Average memory access time is

$$\text{Average memory access time} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

If we assume the hit time is 1 clock cycle independent of block size, then the access time for a 16-byte block in a 4 KB cache is

$$\text{Average memory access time} = 1 + (8.57\% \times 82) = 8.027 \text{ clock cycles}$$

and for a 256-byte block in a 256 KB cache the average memory access time is

$$\text{Average memory access time} = 1 + (0.49\% \times 112) = 1.549 \text{ clock cycles}$$

Larger Block Size: → Increase Miss Penalty
Choose a Block Size Based on AMAT

Block size	Miss penalty	Cache size 4K	16K	64K	256K
16	82	8.027	4.231	2.673	1.894
32	84	**7.082**	3.411	2.134	1.588
64	88	7.160	**3.323**	**1.933**	**1.449**
128	96	8.469	3.659	1.979	1.470
256	112	11.651	4.685	2.288	1.549

Figure B.12 Average memory access time versus block size for five different-sized caches in Figure B.10. Block sizes of 32 and 64 bytes dominate. The smallest average time per cache size is boldfaced.

Average Memory Access Time = Hit Time + Miss Rate * Miss Penalty

Address Translation Issues – VA to PA in Caches – discuss later

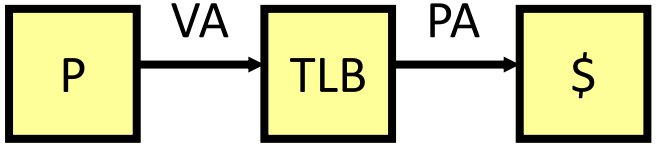
How to Avoid Address Translation during Indexing

- Two tasks: indexing the cache and comparing addresses
- virtually vs. physically addressed cache
  - virtual cache: use virtual address (VA) for the cache
  - physical cache: use physical address (PA) after translating virtual address
- Challenges to virtual cache
  1. **Protection**: page-level protection (RW/RO/Invalid) must be checked
     - It's checked as part of the virtual to physical address translation
     - solution: an addition field to copy the protection information from TLB and check it on every access to the cache
  2. **context switching**: same VA of different processes refer to different PA, requiring the cache to be flushed
     - solution: increase width of cache address tag with process-identifier tag (PID)
  3. **Synonyms or aliases**: two different VA for the same PA
     - inconsistency problem: two copies of the same data in a virtual cache
     - hardware ***antialiasing*** solution: guarantee every cache block a unique PA
       - Alpha 21264: check all possible locations. If one is found, it is invalidated
     - software ***page-coloring*** solution: forcing aliases to share some address bits
       - Sun's Solaris: all aliases must be identical in last 18 bits => no duplicate PA
  4. **I/O**: typically use PA, so need to interact with cache (see Section 5.12)

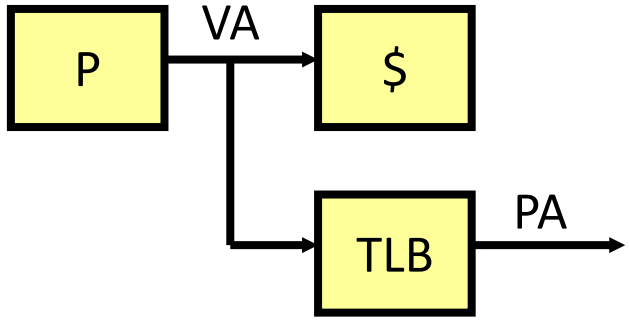
Virtual Memory
- **Later in**
- **The course**

Address Translation in Cache Indexing

- Avoiding address translation in cache indexing
  - Strategy 1: virtually addressed cache



Physical Cache



- Problem 1: Every time on a process context switch, the cache must be flushed; otherwise get false hits across processes
  - Cost: time to flush + "compulsory" misses
  - Solution: add processes identifier tag
- Problem 2: Two different cache entries may hold data for the same physical address → *aliases*

Strategy for Addressed Translation in Indexed Cache

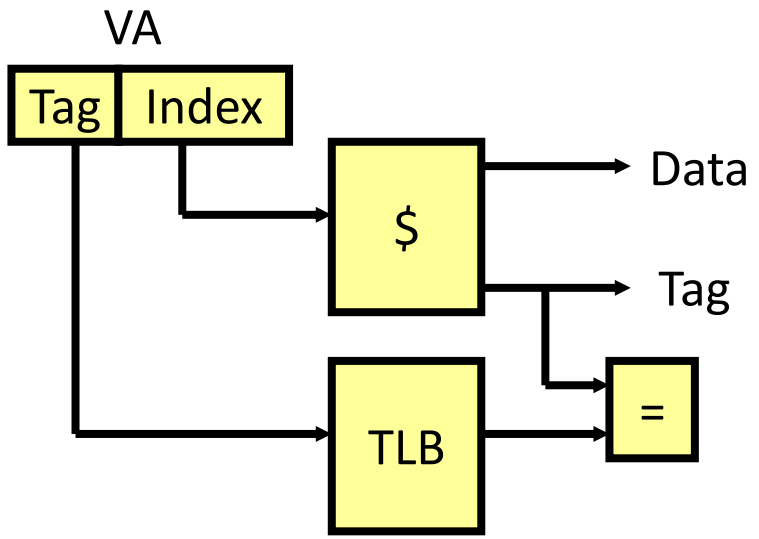
- Avoiding address translation in cache indexing (cont'd)
  - Strategy 1: virtually addressed cache (cont'd)
    - *Aliases* (synonyms):

```
void alias(int *x, int *y) {
        *x = 1;          *y = 0;
}
```

    - Hardware solution: cache controller guarantees that every cache block has unique physical address
    - Software solution: compiler guarantees lower n bits of addresses of aliased variables be the same, where the n bits cover only index+offset fields
      → the 2 variables mapped to same location if direct-mapped cache is used (*page coloring*)

Six Basic Cache Optimizations

- Avoiding address translation in cache indexing (cont'd)
  - Strategy 2: physically mapped, virtually tagged
    - Index+offset fields are same in virtual as well as physical addresses (no need to be translated), e.g. within a VM page
    - Can start cache tag access in parallel with translation so that physical tag can be compared
    - Problem: limit cache size to a VM page
      - Use higher associativity
      - Use page coloring (see previous page)



Virtual Memory Concept

Motivation (1)

- Computers these days run more than one program at once
- Problem:
  - Program A puts its variable X at address 0x1000
  - Program B puts its variably Q at address 0x1000
  - Conflict!
- Solution:
  - Allow each running program to have its own "view" of memory (e.g. "my address 0x1000 is different from your address 0x1000")

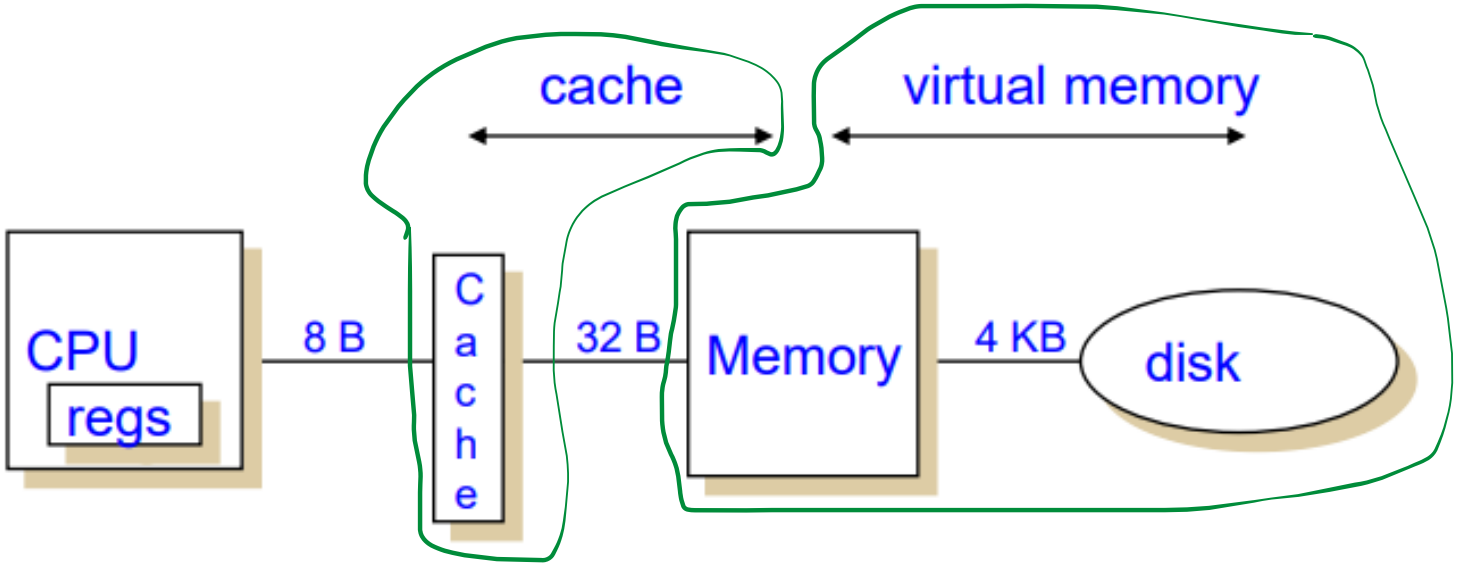
- How? Add a layer of **indirection** to memory addressing: **Virtual memory paging**

Motivation (2)

Motivations for Virtual Memory

- **Use Physical DRAM as a Cache for the Disk**
  - Address space of a process can exceed physical memory size
  - Sum of address spaces of multiple processes can exceed physical memory
- **Simplify Memory Management**
  - Multiple processes resident in main memory
    - Each process with its own address space
  - Only "active" code and data is actually in memory
    - Allocate more memory to process as needed
- **Provide Protection**
  - One process can't interfere with another
    - because they operate in different address spaces
  - User process cannot access privileged information
    - different sections of address spaces have different permissions

Levels in Memory Hierarchy



	Register	Cache	Memory	Disk Memory
size:	32 B	32 KB-4MB	1024 MB	100 GB
speed:	1 ns	2 ns	30 ns	8 ms
$/Mbyte:		$125/MB	$0.20/MB	$0.001/MB
line size:	8 B	32 B	4 KB	

larger, slower, cheaper →

Benefits of Virtual Memory

Paging benefits:

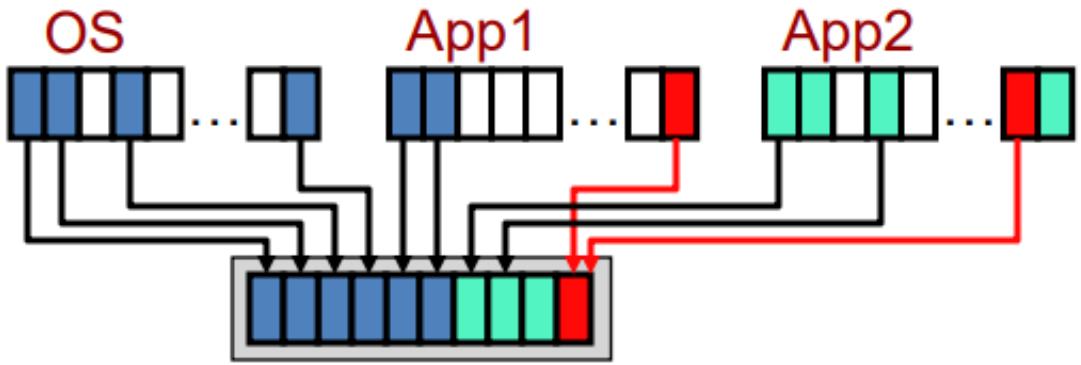
- **Simpler programs**:
  - We "virtualize" memory addresses, so every program believes it has a full 2^{32} or 2^{64} byte address space
- **Easier sharing**:
  - Processes can "share" memory, i.e. have virtual addresses that map to the same physical addresses

Swapping benefits:

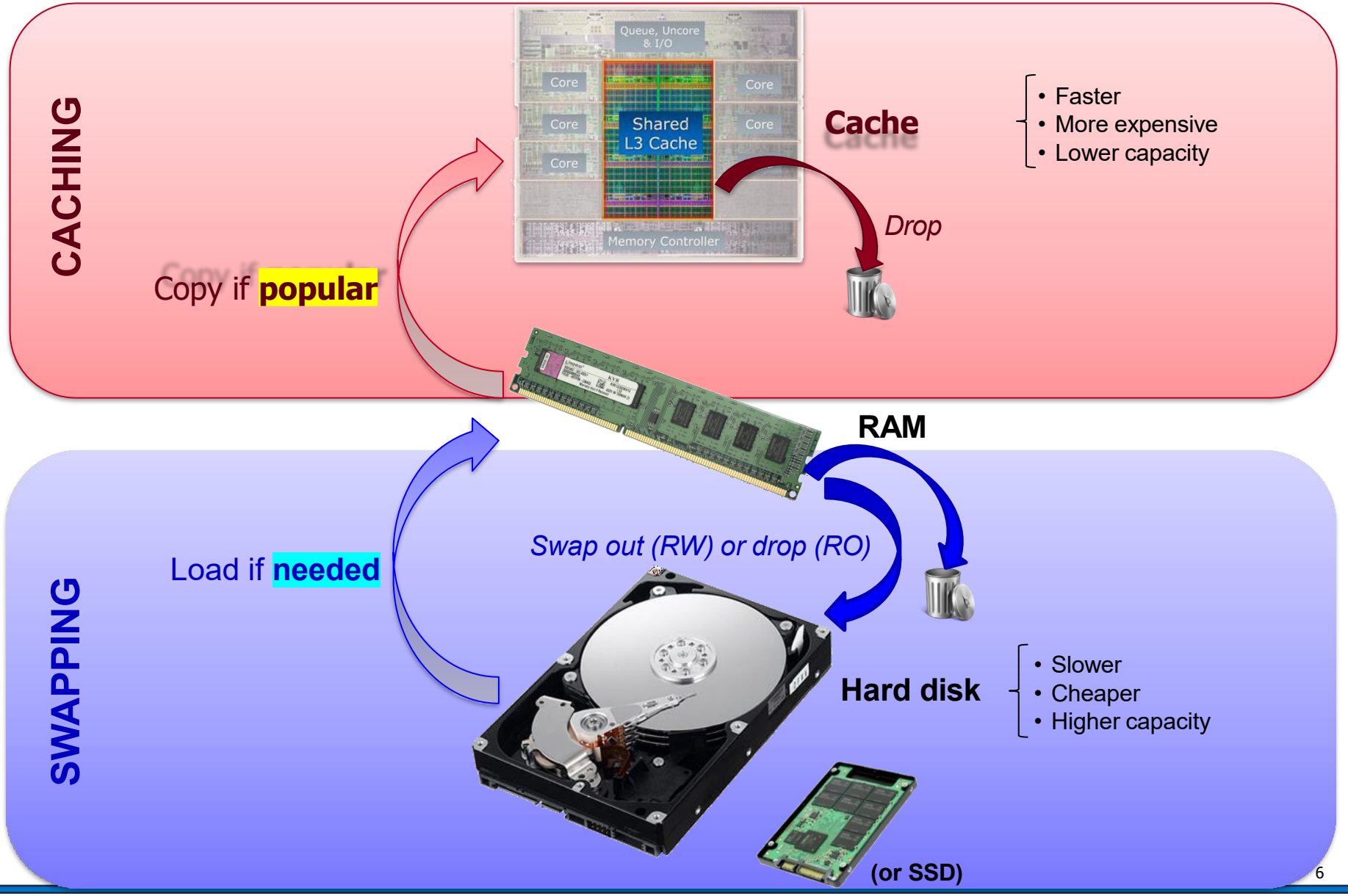
- **Bigger programs**:
  - Not just bigger than free memory, bigger than AVAILABLE memory!
- **More programs**:
  - Only part of each program loaded, so more can be loaded at once
- **Faster multitasking**:
  - Less effort to load or swap processes

Uses of Virtual Memory

- **Isolation** and **Multi-programming (Memory Management)**
  - Each app thinks it has 2^N B of memory that starts @ 0
  - Apps can't read/write each other's memory
    - Can't even address the other program's memory!
- **Protection**
  - Each page has read/write/execute permission set by OS
  - Enforced by hardware
- **Inter-process communication**
  - Map same physical pages into multiple virtual address spaces
  - Or share files via the UNIX `mmap()` call



Caching and Swapping in Virtual Memory



High Level VM Operation Concept

